INTEGRATIVE PROGRAMMING — FINAL PROJECT DEFENSE

Shopee Clone: 5-to-7 Minute Demo Script

Complete Presentation Guide • UI Walkthrough, Code Explanations & Oral Defense Defense Strategy

ARCHITECTURE	TARGET DURATION	SECURITY STACK	VALIDATION
React 19 + Express 5 + MySQL	5 Minutes (Max 7 mins)	bcrypt + JWT + Parameterized SQL	Zod + React Hook Form

Time	Segment	Core Technical Highlight
0:00 – 1:00	Introduction & Architecture	3-tier architecture, connection pooling, client-server decoupling
1:00 – 2:30	Authentication & Security	bcrypt password hashing (10 salt rounds), JWT token issuance & claims
2:30 – 3:45	Catalog, Search & Custom Hook	Reusable `useApiData` hook, cleanup flag, dynamic WHERE clause
3:45 – 5:15	Cart & Transactional Checkout	ACID Transactions (`beginTransaction/commit`), `FOR UPDATE` locking, server prices
5:15 – 6:15	Order Management & Data Privacy	Protected routes (`authenticateToken`), data ownership (`WHERE user_id = req.user.id`)
6:15 – 7:00	Conclusion & Q&A Defense	Summary of engineering decisions and invitation for professor queries

1. Introduction & Tech Stack Overview

0:00 – 1:00

SCREEN ACTION

Open browser at `http://localhost:5173` showing the Sign-In card. Have VS Code ready in the background with `server.js` and `App.tsx` open.

SPOKEN SCRIPT (WORD-FOR-WORD)

"Good day, Professor! Today we are presenting our Integrative Programming final project: a full-stack **Shopee Clone E-Commerce Application** with user authentication, catalog management, and a transactional checkout system.

Our system is built on a clean 3-tier architecture:

- **Frontend:** React 19 with TypeScript, styled with custom Shopee-inspired CSS and bundled with Vite.
- **Backend:** Node.js with Express 5, implementing a secure RESTful API.
- **Database:** MySQL relational database connected via connection pooling using `mysql2/promise`.
- **Validation & Security:** Client-side Zod schemas with React Hook Form, paired with server-side bcrypt password hashing and stateless JSON Web Tokens.

Let us walk you through the system workflow and the code behind it."

2. Authentication (Sign-Up, Login, bcrypt & JWT)

1:00 – 2:30

SCREEN ACTION

1. Click "*New to Shopee? Sign Up*".
2. Type an invalid email (e.g. `test`) or short password to show the red validation message.
3. Type valid credentials and click **"Sign Up"** (or log in with an existing account). Show that it immediately logs in and displays the user's name in the header.

CODE LOCATION IN VS CODE

`server/server.js lines 107–114 (signup handler) & lines 138–151 (login handler)`

SPOKEN SCRIPT

"Starting with Authentication: notice when I type an invalid email or short password, our client-side **Zod schema** in `schemas.ts` catches it instantly on blur before any network request is sent.

Behind the scenes in `server.js`:

1. When the user registers, we hash the password with **bcrypt** using 10 salt rounds (`bcrypt.hash`). Plain passwords are never stored in MySQL.
2. During login, `bcrypt.compare` verifies the entered password against the stored database hash.
3. Once verified, the server issues a **JSON Web Token (JWT)** signed with our secret key. It contains the user's ID, name, and email and expires in 24 hours.
4. The client stores this token in `localStorage`, attaching it to the `Authorization: Bearer` header on all protected API calls."

3. Product Catalog, Search & Custom Reusable Hook

2:30 – 3:45

SCREEN ACTION

1. On the **"Mall Products"** tab, type "*keyboard*" in search and click **Search / Filter**.
2. Clear it, enter Min Price: `50`, Max Price: `150`, click Search. Show dynamic grid updates.
3. Click **Clear** to restore the catalog.

CODE LOCATION IN VS CODE

`client/src/hooks/useApiData.ts lines 12–57 & server/server.js lines 192–215`

SPOKEN SCRIPT

"Here on the storefront, products are fetched dynamically from our MySQL `products` table.

Instead of copy-pasting data fetching in multiple components, we built a reusable custom React hook called `useApiData`. It encapsulates:

- The `data` array, `loading` state, and `error` state.
- A `refresh()` callback to trigger data re-fetching without reloading the entire page.
- And a cleanup flag (`isCurrent = false`) to discard stale responses if the user rapidly searches.

On the backend, `listProductsHandler` dynamically constructs the SQL `WHERE` clause using parameterized `?` placeholders, keeping the API fast and safe from SQL injection."

4. Cart & Transactional Checkout (The Main Star!)

3:45 – 5:15

SCREEN ACTION

1. Click **"+ Add"** on two products. Point out the bottom-right confirmation toast.
2. Switch to **"My Cart"** tab. Increase quantity on one item.
3. Fill out the Philippine address form: *123 Mabini St.*, *Quezon City*, *Metro Manila*, *1100*, Phone: *09171234567*.
4. Click **"Place Order"**. The cart clears, toast pops up, and it automatically switches to **"My Orders"**.

CODE LOCATION IN VS CODE

`server/server.js lines 300–353 (createOrderHandler transaction)`

SPOKEN SCRIPT

"Now for the checkout process — this is the most critical part of our system architecture.

Notice two important engineering decisions we made:

1. **Server-Side Price Validation:** The server completely ignores prices sent by the browser. It recalculates the total directly from current database product records to prevent client-side price tampering.
2. **ACID MySQL Transactions with Concurrency Control:** In `server.js`, you can see `connection.beginTransaction()`. For every item, we execute `SELECT ... FOR UPDATE`. This locks the database row so two concurrent buyers cannot buy the last available item simultaneously.
   The server inserts the order header into `checkout_orders`, records item snapshots into `order_items`, and decrements stock in `products`. If any item is out of stock, it executes `rollback()`. If everything succeeds, it executes `commit()`. All or nothing."

5. Order History & Data Ownership

5:15 – 6:15

SCREEN ACTION

1. Show the placed order at the top of **"My Orders"**.
2. Change order status from "*Pending*" to "*Shipped*" using the dropdown.
3. Click **"My Account Settings"** tab, edit the display name (e.g. "*John Developer*"), click Save.
4. Show the header welcome message updating immediately.

CODE LOCATION IN VS CODE

`server/server.js line 360 (WHERE user_id = req.user.id) & line 438 (re-issued token)`

SPOKEN SCRIPT

"Here in Order History, users can track and update the status of their orders.

To enforce **Data Ownership and User Privacy**, all order routes are protected by our `authenticateToken` middleware. Queries explicitly filter by `WHERE user_id = req.user.id`. Even if someone knows another user's order ID, the API will never expose or alter orders belonging to another account.

Under Account Settings, updating the user's name updates MySQL and immediately returns a freshly re-signed JWT token containing the new name, updating the UI header seamlessly without requiring a re-login."

6. Conclusion & Open for Defense

6:15 – 7:00

SPOKEN SCRIPT

"To summarize our implementation:

- A fully integrated full-stack web application using React 19, Node.js Express, and MySQL.
- Enterprise-grade security: bcrypt password hashing, stateless JWT authentication, and parameterized SQL queries.
- Data integrity through ACID database transactions with row-level locking (`FOR UPDATE`) and dual-layer validation with Zod.

Thank you very much, Professor! We are now open for your questions."

🎯 Professor Oral Defense Cheat Sheet (Fast Answers)

Likely Professor Question	Precise Technical Answer to Give
Why use Database Transactions during checkout?	"To guarantee the Atomicity property of ACID. If product stock deduction fails after inserting the order header, `connection.rollback()` cancels everything so our database is never left in an inconsistent state."
What does `FOR UPDATE` do?	"It applies an exclusive row-level lock in MySQL during the checkout transaction. This prevents Race Conditions and overselling when two users try to purchase the last unit at the exact same millisecond."
Can a user spoof product prices from Chrome DevTools?	"No. Our backend enforces Server-Side Price Calculation. It ignores client prices and re-reads the official price directly from the MySQL `products` table."
How does login verify a password if it's hashed?	"We use `bcrypt.compare()`. It hashes the entered password using the salt stored in the hash and compares the result, without ever needing to decrypt the stored hash."
Why use JWT instead of traditional sessions?	"JWTs are stateless and self-contained. The server does not need to maintain session memory or hit a session database for every request; it simply verifies the cryptographic signature with our secret."
How do you prevent SQL Injection?	"All SQL queries use parameterized prepared statements with `?` placeholders. User input is treated strictly as data literals, never as executable SQL commands."

💡 **Presentation Tips for Tomorrow:**

1. Ensure MySQL, backend (`npm run dev --prefix server`), and client (`npm run dev --prefix client`) are already running before you enter the room.
2. Have one test account already registered so you don't waste time typing during the intro.
3. If the professor asks you to show the database, open MySQL Workbench / HeidiSQL and show `checkout_orders` and `order_items` with the foreign key relation.